

Real-Time Systems – Design Project 2026

Combined assessment value is 50% for both EEET2166 and EEET1262

Tentative Due Dates:

- Team Formation: ASAP or 9am Wednesday 2nd Sep 2026 (semester break).
- Initial Design Report Due Date: 9am Friday 18th Sep 2026 (week 8).
- Final Report and Software Due: 9am Friday 16th Oct 2026 (week 12).
- Implementation Demonstration: 16th Oct 2026 (by appointment in week 12).

Due dates provide above are not the official dates, rather the official dates are defined by the settings on Canvas. Hence you should always refer to the date on canvas for the most up-to-date due dates for each assessment item.

Aim: To work through the process of designing a real-time system and demonstrate an implementation. Complexity of design will achieve a higher grade only if implemented in a sensible and justifiable manner – Your claims will need to back-up by evidence during demonstration.

You are to design a Traffic Light control system according to the requirements given in this document (pages 2-3) and implement a "proof of concept" solution.

Project Overview

1. Form a group of up to four people (can be a mix from different lab classes or EEET2166 and EEET1262 students as the same marking guide applies to both cohorts).
2. Each group must develop an initial design for the Traffic Light control system and submit an initial design report (an outline of the expected report contents is given on page 5).
3. Develop a "proof-of-concept" implementation using the QNX Real-Time Operating System. Each group must demonstrate their solution in week 12. Demonstrations will be at a time scheduled by the lecturer at the end of the semester that does not clash with university or school commitments such as Capstone presentations. If this occurs the date of the demonstration may be extending into week 13.
4. Submit the final report (the required contents of this report are given on page 6) and the final software.

Submission Requirements

Marks allocation: This project is worth 50% of the marks for the course. The marks allocation is broken down as the following assessment items:

- Initial Design Report 5%
- Final Report 15%
- Implementation 30% (this includes the software demonstration)

Groups: Ideally, groups of 3 people will need to be formed. Each group is to appoint a team leader who will be responsible for all communication and submission tasks. The first task will be to add the team members to the group formation spreadsheet:

[2026_RT_group_project_formation.xlsx](#)

Reports: Reports must be submitted by uploading the document to Canvas, complete with coversheet and declaration that the work being submitted is that of the enrolled student group members. Only one group member needs to submit the electronic copy.

A breakdown of each group member contributions also needs to be provided. **This should be included as an Appendix and should be signed by all members of the group if an unequal weighting is to be considered during assessment.**

Implementation: Implementation must be done using the QNX Real-Time operating system with software written in C or C++ and must use the QNX supported **interprocess communications (IPC) and synchronisation primitives**.

Software submission: The software developed by the team must be submitted in electronic form (via Canvas). Only one group member needs to submit the software. It should be submitted as a *.zip file at the time the final report is submitted by the due date/time.

Demonstration: Each team will be required to formally demonstrate their solution. All team members are required to demonstrate the part of the project they worked on. Demonstrations will be limited to 15 minutes maximum with a 10-15 minute question and answer session to follow. You must address all aspects within this time frame.

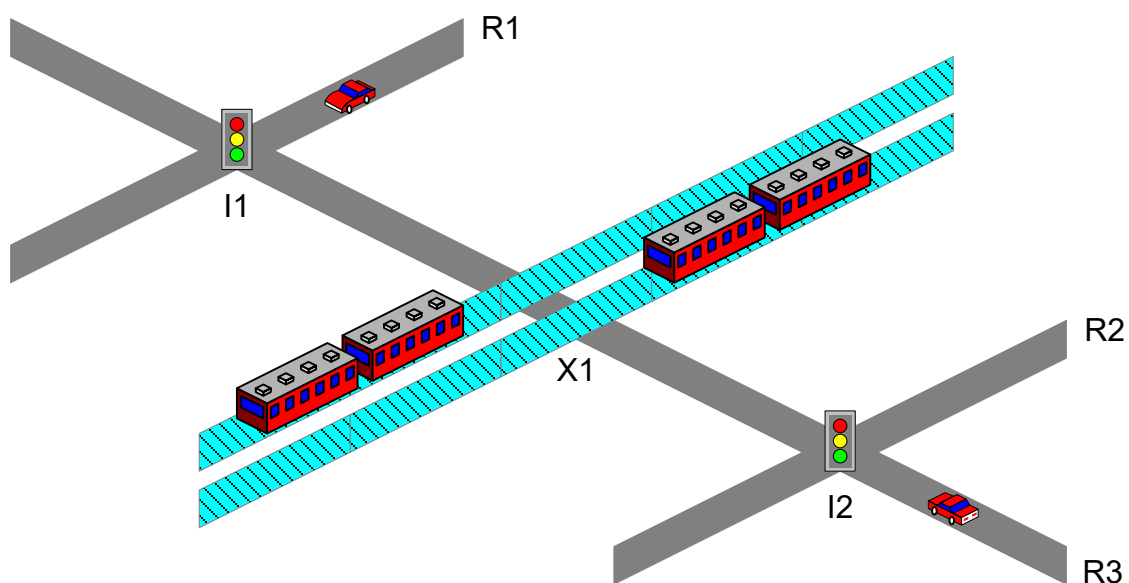
Any queries regarding this project should be directed to the lecturer, Samuel Ippolito, via email: samuel.ippolito@rmit.edu.au Put "EEET1262/2166" in subject heading.

Traffic Light Controller

Use the real-time systems design methodologies outlined in lectures to develop and design a real-time system for traffic light control according to the specification that follow. You will be required to develop a functioning "proof-of-concept" implementation using multiple QNX nodes. Use Design Diagrams (including UML variants) to document your design.

Traffic Lights Control System Requirements

The system is to control at least two sets of traffic lights and a set of boom gates for a busy train line at intersections as given in the following diagram. This is the minimum level of complexity to achieve a Pass grade.



In the example above, there is a road (R3 going from top-left to bottom-right of diagram, but it actually goes in a North-South direction) along which there are two road intersections (I1 and I2) and a railway crossing (X1). The crossing roads at each intersection (R1 and R2) each go in an East-West direction. All roads (East-West and North-South) carry two-way traffic, have two lanes in each direction, may have turn lanes at each intersection (I1 and I2), and may have pedestrian

crossing lights at each intersection (I1 and I2). R1 and R3 are major roads with a high level of traffic, whereas R2 (whilst still a major road) has only a moderate level of traffic.

The railway crossing has boom gates and red flashing lights that operate whenever a train crosses the road. There are two train lines, one for trains travelling each direction. The time between trains varies from frequent (as little as 2 minutes apart) at peak hour to infrequent (about every 20 minutes) at night (~10:00pm) - All night trains run on Friday and Saturday nights. The traffic light system can sense the state of the railway crossing (but has no control over it). The train should receive a red light if there are any issues with the boom gates – thus indicating an error which should be reported to the control room.

Each intersection/pedestrian crossing has a local controller for the lights at the intersection which operate continuously (except for hardware failure). Each local controller also communicates with a central controller (but should continue to operate if either communication link fails or the central controller goes offline).

The two intersection controllers (I1 and I2) operate independently, however can receive commands from a central controller to determine the light sequence pattern that they should follow for different times of the day. However, both intersections must sense the state of the railway crossing (X1) and sequence their set of lights for R3 to red when a train is crossing at X1, whilst still trying to accommodate traffic on all three roads. The support for pedestrian buttons and lights should also be considered for grades above a Credit level.

There is a central control room with a central controller, which monitors the traffic lights and displays the status, and light settings received from the individual intersections. Therefore, each local controller will send a status message to the central controller whenever any of the lights at that intersection change state, and act on any commands received from the central controller.

Note that the central controller does not directly control the individual lights at an intersection - that is always done by the local controller and must be maintained even in the event that the controller cannot communicate with an intersection or another controller. A full implementation would also have a train line controller that handles the communication link between the traffic central controller and the train lines.

Modelling your design on a real intersection:

Below is an example intersection in Melbourne which might help with breaking down the problem. You should at least attempt to base your design on a real intersection - this may help with you formulating your justifications and requirements. You can include tram line signalling if needed too.



Image source: <https://www.google.com.au/maps/@-37.705977,144.9167887,18z>

Note: This image was taken before the state government removed the train line in 2021

By measuring the distance between the traffic light intersections and train crossing you can start to determine the timing requirements and traffic light sequences that will be needed in-order to clear the intersections and avoid traffic backing up. By making some realistic assumptions, the side roads can be eliminated from the design as they are not important. As an example, during peak time with trains approaching from both sides, it can take between 3 to 10 minutes to follow the path outlined by the red arrows if there are pedestrians trying to cross Glenroy Rd or Hartington St.

Please note: The example intersection is only provided to illustrate a real-life example. You should choose your own model intersection as of 2021, this rail crossing was removed by the Victorian government.

Possible light sequence patterns for each intersection local controller (set by a command from the central controller) include:

- Fixed timing – commonly used during peak hours
- Sensor driven (react to local car sensors & any pedestrian buttons) – commonly used in off-peak times.
- Advanced: updateable sequence structure or timing requests from central controller.

The central controller may send a command to change the sequence a few times a day (e.g. depending on the time of day).

Operators in the central control room will occasionally send commands to the intersections, boom gate control and train approach signals system.

The central control room may also initiate override commands that are sent to the individual intersections to deal with exceptional situations (e.g. to provide a clear path for a visiting dignitary).

At all times the lights at each intersection must exhibit the “expected” behaviour of real traffic lights, however the timing between state changes may be speed up for practical and demonstration reasons.

These specifications do not describe every possible feature of the intersections, leaving some room for variations in solutions (e.g. pedestrian lights, right turn arrows etc at the intersections).

You will need to simulate sensor events in your implementation e.g. using key presses (this will require an additional process or thread to wait for the key presses). You are free to use console input, the provided Beaglebone LCD/keypads in the lab (if available), or GPIO developed on your own Raspberry Pi 4, DE10-nano or Beaglebone targets. Otherwise, you would need to use the consoles or Telnet sessions to multiple _x86 QNX Target VMs, depending on the complexity of the solution your team is aiming to achieve. See the accompanying project rubric to get an idea of the different levels of complexity that are expected for different grade levels.

Note that a demonstration using only three QNX node will attract a Pass to Credit level (e.g. I1 running on one computer, I2 running on a second node and a third QNX node acting as the central controller or train controller) as define in the rubric. In all cases, it is expected that your solution will have three or more QNX nodes with separate processes/threads on each node to handle the features such as pedestrian / sensor inputs and/or communication links, with another process used to display the traffic light states.

You are not required to have a graphical user interface; it is sufficient to display the state of each intersection by writing simple text messages to a terminal window or via and LCD attached to the Beaglebone or a customised hardware display on your own target/s. You are encouraged to use the provided Beaglebone LEDs and/or LCD to display states (if lab hardware is made available) or your own interface hardware. You can also make use of the persistent file system on some targets (i.e. Beaglebone supports this feature) under the /fs directory. You can store test vectors or debug information for offline analysis, or any other feature you think is suitable.

You are free to make any reasonable assumptions necessary to proceed with the design as long as you outline them upfront, however it is a good idea to justify your decision on a simplified real-life traffic light intersection.

As a rough guide, according to the rubric, a group of 4 students aiming for High Distinction grade level would propose and deliver a system using 5 or more QNX nodes with custom hardware with IPC implemented with native message passing techniques and a host of features such as:

updateable pattern sequencing; external sensors and displays; custom hardware; separate nodes for each intersection and controller; appropriate synchronization primitives; timers and software/hardware interrupts; excellent code management and commenting; and host of well thought-out features and well timed demonstration/presentation. Obviously, there is room for movement as a given team might focus more highly on certain features over other features or implementation methods.

Initial Design Report

The initial design report should show the steps you went through in the design process leading to your initial design. Expected design documentation includes:

- Your interpretation of the problem, and any assumptions you made about how the system will operate.
- A short discussion as to what grade level you intend to deliver your solution too. Note: you will not be held to this level, it will just be used to provide feedback at the appropriate level. You can change, drop, or add features to your final design/submission as you see fit to target a lower or higher level.
- A detailed diagram of the intersection layouts for your design indicating the various lights and sensors (which depend on the intersection features supported, such as right-turn arrows and pedestrian crossings).
- State Charts describing the lights behaviour at each intersection.
- Use Case diagram(s) and use case scenarios including relevant State Chart(s) and written specifications.
- Behavioural specifications such as sequence diagram(s) or collaboration diagrams may be included.
- High level implementation diagrams especially a task architecture diagram.
- Overview of messages sent between tasks (type, purpose and outline of message content).
- Add explanatory notes particularly to diagrams, to ensure that your design is fully documented and clearly explained.

Diagrams must be clearly labelled and numbered (this will help if you refer to any diagram in the initial report from the final report).

Report must have a Table of Contents, professional layout and proper page headers and footers. It should also include a contribution declaration from each team member and coverpage.

See page 1 of this document (under Implementation) for further details. You should also look at the provided RTS Project Initial Design Marking Sheet PDF file.

Software Demonstration (Implementation)

Each team will be required to formally demonstrate their solution as part of the Implementation and Technical Demonstration assessment item. This is by far the most important item of the course. All team members are required to attend and present an aspect of the solution. Demonstrations will be scheduled by the lecturer in week 12. Each demonstration will be preformed in 2 parts and go for no more than 30 minutes in total. The first part will be a student driven demonstration of all features that will go for no more than 15 minutes. The remaining 10 to 15 minutes will be a Question and Answer (Q&A) session driven by the assessor. During the Q&A session you might be required to re-demonstrate a given aspect of your solution under a certain situation to prove your claims or discuss aspects of the code.

NOTE: All demonstrations will be held during a face-to-face meeting in RMIT 10.09.11 laboratory. It is recommended that each team prepare a demonstration that goes for no more than 15 minutes long for part 1 of the software demonstration assessment. All features and scenarios must be

discussed and demonstrated within this time frame. Teams may demonstrate their system using a pre-recorded video to maximise the time, however this is up to the individual teams as a live demo will be required for part 2 of the demonstration. All team members must take part in the meeting and present/answer questions for part 2 of the assessment session. In all cases, a live demonstration of the teams solution will be required during the second part of the assessment as the assessor will ask each team to demonstrate specific situations live on a fully running system. Teams that do not provide a live demo to backup each of the claims made in part 1 of the demonstration will be penalised if the solution does not perform as claimed when demonstrated live in part 2. Both part 1 and part 2 of the meeting will go no longer than 30 minutes combined for each team.

During the demonstration, you will be assessed on your professionalism as well as your technical justifications and solution. Remember: An unjustifiable and overly complex system does not equal good marks. Equally an overly simplified design/implementation will not achieve a good mark and may result in less than 50%. You are expected to demonstrate where you used any interprocess communications (IPC) and synchronisation primitives, as well as how you tested your solution.

To be eligible to attend the demonstrate session, all code and a copy of the final report must first be uploaded to Canvas by the deadlines defined on canvas. Further details are provided on page 1 and 2 of this document under the Implementation, Software Submission and Demonstration subheadings. You should also look at the provided RTS Project Review Checklist and RTS Project Demonstration Rubric PDF files.

Final Design Report

The report must follow the format given here, including detailing your final design, highlighting any changes from your initial design and discussing why the changes were made. The final report should bridge the logic gaps over the initial design report.

Note that for diagrams which are unchanged from the Initial Design Report there can be repeat in the Final Report - just make a reference to it. You will not lose marks if you make amendments to your initial design as long as they are justified, and the benefits are clearly explained. It is expected that there will be some changes implements between the initial design report and the final report.

In this report you must address the following points:

1. Restate the problem as you now understand it in your own words. State what assumptions you made and how you interpreted what you had to do. You should also state what constitutes a solution to the problem. Extract the design requirements and benefits of your team's solution!
2. Provide a formal statement of your solution to the problem together with explanations, proofs and other evidence to justify your solution. This must include detailed design diagrams, message formats and message protocol descriptions. An evaluation of the solution should be given and, where appropriate, you should suggest how your results could be generalised and possible directions in which future work could proceed.
3. Report on the solution process that you went through to arrive at your result (from initial design through to final implementation). Describe your attempts to solve the problems. Show how you started off, how your understanding of the problem developed, what you tried at various stages and any important insights and breakthroughs which occurred as you sought solutions to problems that arose. You should highlight, with hindsight, the problem-solving strategies, any mathematical techniques, formal methods, testing methods, and computer assistance you tried. Any rough notes made in exploring the problem may be included as an appendix.
4. Discuss the safety and fault tolerance requirements for this system and what you would do to address these requirements in a real system. What happens if a node fails? You should include a comprehensive test plan.
5. Reflect on the project. Think about what you have learned by doing the project and reflect on the original objectives of the project compared with what you finally achieved.

Suggestions for improvements that you feel will enhance the project for students studying this subject in the future will be greatly appreciated.

6. Give a brief acknowledgment of the ideas or other help provided by other students, specialist information sources, and so on. Include any references actually found to be useful in the solution of the problem.
7. Include any appendices that may help the reader understand important details of your report (e.g. extended test plans/scenarios or secondary diagrams, etc.). Appendices should only be included if they have been specifically referred to in the text. They should be numbered consecutively and each should bear a title.

The assessment breakdown for the above points in your report is:

$$10 + 30 + 20 + 15 + 15 + 5 + 5 = 100$$

Report must have a Table of Contents, professional layout and proper page headers and footers. It should also include a contribution declaration from each team member and coverpage.

Remember: A well-structured, short and concise report is better than a long-winded document. Use your appendixes to make the main document easy to read. Significant diagrams should be within the main report and accompanying text. All diagrams should be referenced by the main text.

Further details are provided on page 1 of this document under Reports and Software Submission. You should also look at the provided RTS Project Final Report Marking Sheet PDF file.

End of document!